Observability Deep Dive

Correlating Logs Across Azure Monitor, App Insights & Distributed Services

Monika Mundra

Technical Architect, Hexaware

- MS certified AI business transformation Leader
- Azure Certified Architect
- PMP certified Professional

Microsoft Build //localhost:newyork

6/9/2026

8:00-9:00		🍳 Check In & Breakfast 🍳
9:00-9:05		🗽 Kickoff & Welcome 🥳
9:05-9:40	Peter Ward	***Keynote***: The hidden value of asking better questions with AI projects
9:40-10:15	Dave Goad	Moving Beyond Vibe Coding to Spec Driven AI Native Development in the Enterprise
10:15:10:35		🧉 Break 🧉
10:35-11:10	Manpreet Singh	Toolkit for building Agents: M365 Copilot, Studio & SharePoint in Action
11:10-11:45	Vladimir Gusarov	Your Backlog Is Lying: Enforce It with PR Checks
11:45-12:20	Preeti Gupta	Data Enablement in Regulated Finance: Moving Beyond Access to Outcomes
12:20-12:40		🧉 Break 🧉
12:40-13:15	Peter Smulovics	From UDDI to MCP – How Did Finding APIs Change in the Last 30 Years
13:15-13:50	David Patrick	From Zero to ChatGPT: Building Your Own AI Assistant with Azure AI
13:50-14:25	Monika Mundra	Observability Deep Dive: Correlating logs across azure monitor, app insights and distributed service
14:25-15:25		🍴 Lunch 🍴
15:25-16:00	⭐ Jason Beres ⭐	***Sponsor Presentation***: Building Enterprise Apps with AI, MCP and Components
16:00-16:35	Abhijeet Jadhav	Copilot Studio to Azure AI Foundry: A Framework for Knowing When — and How — to Scale
16:35-17:00		🎙️ MVP Panel 🎙️
17:00-		🛜 Networking / Mingle 🛜

Session Agenda

The Observability Gap

Why correlation fails in production

Azure Monitor Architecture

Data plane, control plane & routing

Application Insights

SDK, sampling & telemetry correlation

Distributed Tracing

Trace IDs, W3C TraceContext, OpenTelemetry

KQL Correlation Queries

Cross-workspace joins & dashboards

Production Patterns & Q&A

Alerting, runbooks

The Observability Gap

Why distributed systems are hard to debug

Siloed Logs

App logs live in App Insights; infra metrics in Azure Monitor; no correlation key

Missing Context

A 500 error in API Gateway gives no trace to downstream service call failures

Sampling Gaps

Adaptive sampling drops 95% of telemetry — the 5% kept may miss the failing request

73%

of incidents take >1 hr to diagnose

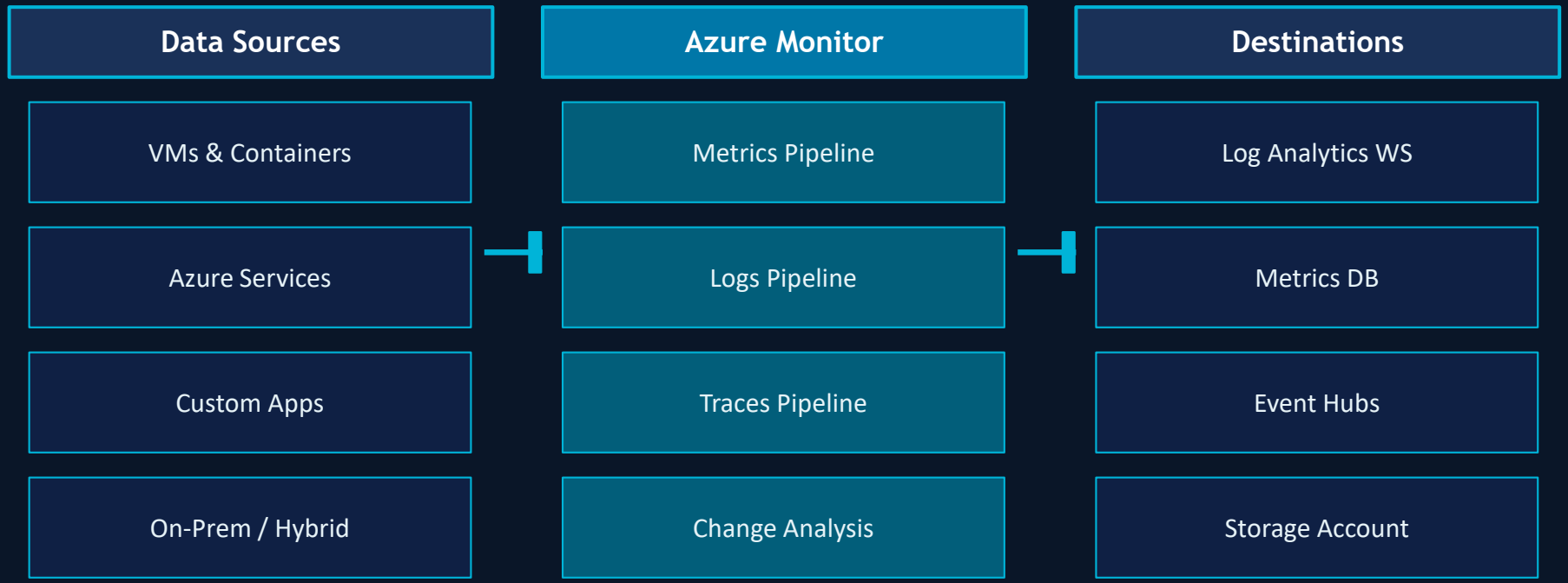
5×

more context needed to resolve P1 issues

$5.6M

avg. annual cost of downtime per org

Azure Monitor: The Observability Backbone



💡 *All pipelines converge in Log Analytics — enabling cross-signal correlation via KQL*

Data Collection Rules (DCR): Route Everything

What DCRs Control

- Which data streams are collected
- Transformation & filtering (KQL-based)
- Routing to multiple destinations
- Sensitive data masking before storage
- Cost control via column projection

DCR Transformation (KQL)

```
source
| where Level != 'Verbose'
| extend
    corrId = tostring(
      Properties.correlationId),
    svcName = tostring(
      Properties.serviceName)
| project TimeGenerated,
    Level, Message,
    corrId, svcName
```

Application Insights: Types of Telemetry

Requests

HTTP/gRPC duration, success, result codes

Dependencies

SQL, HTTP, Service Bus call tracking

Exceptions

Stack traces with full correlation context

Traces

ILogger / Serilog / NLog enrichment

Events

Custom business events with properties

Metrics

Pre-aggregated & live metrics stream

All telemetry types share operation_Id — the master correlation key linking every signal

Sampling Strategies: Keep What Matters

Sampling Type Comparison

Type	Rate	Preserves Traces?	Use When
Fixed-Rate	Configurable	✓ Yes	Low-volume APIs
Adaptive	Dynamic	Partial	High-volume services
Ingestion	Server-side	✗ No	Cost control only
Tail-based (OTel)	Dynamic	✓ Yes	Error-aware sampling

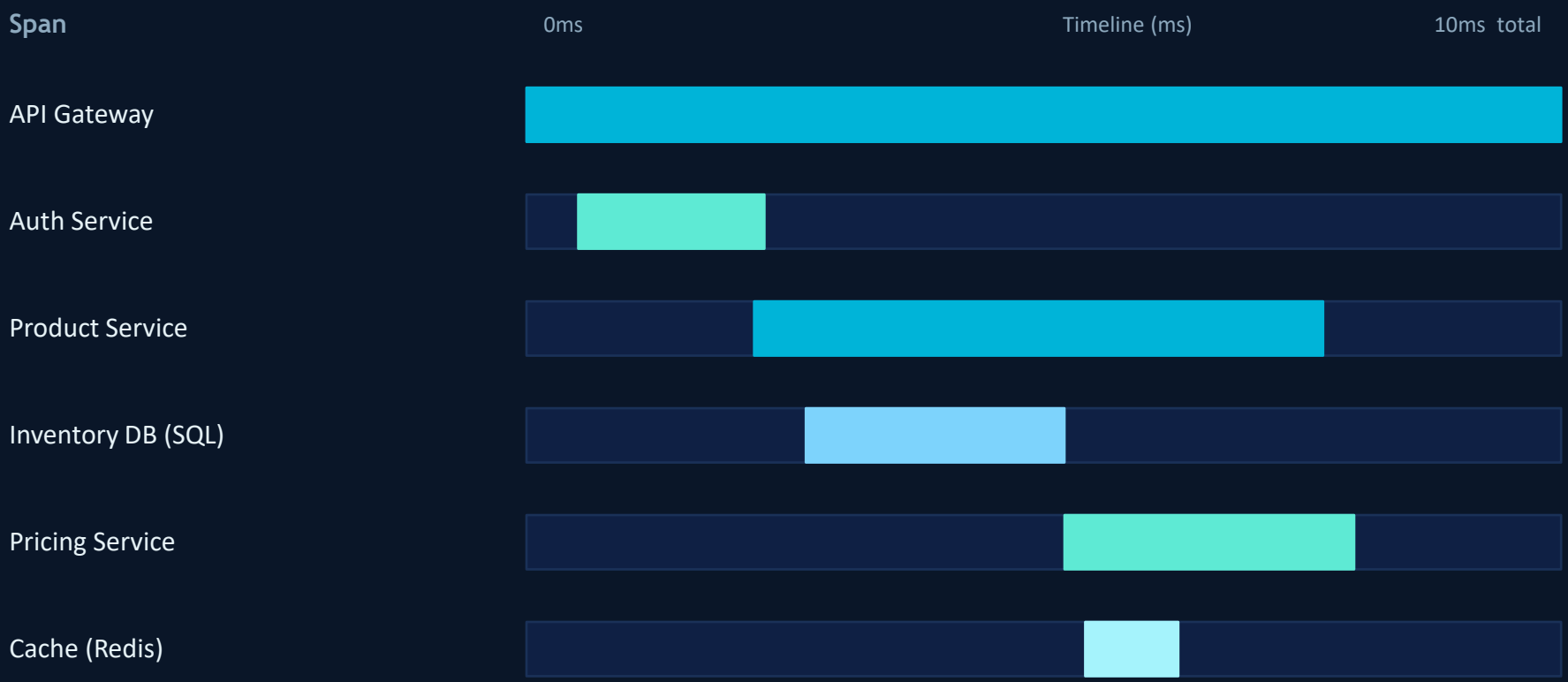
Recommendation

1. Use tail-based sampling with OpenTelemetry for error & latency-triggered full trace capture

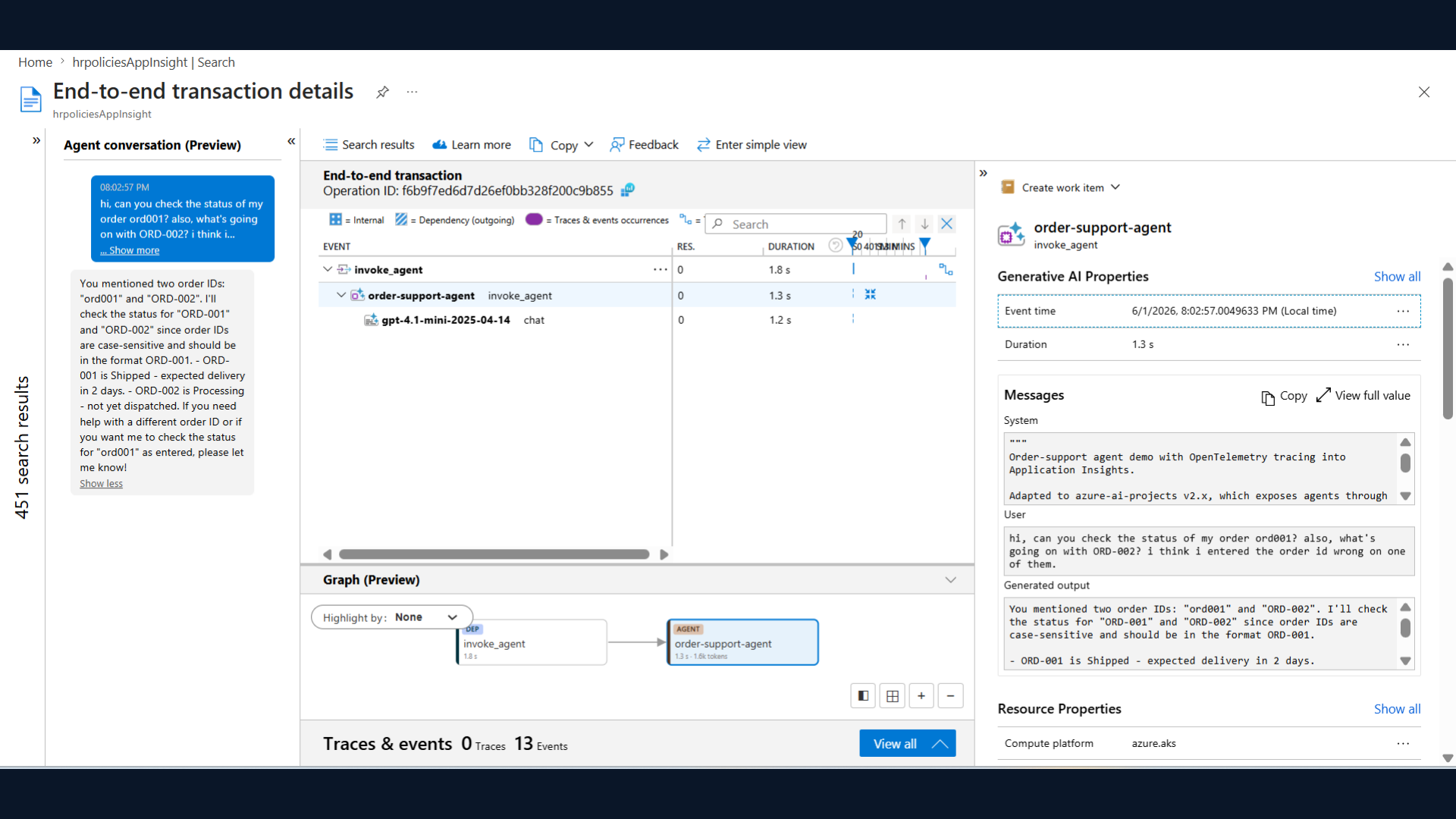
2. Always exempt exceptions and slow requests from down-sampling

3. Propagate samplingRate to enable statistical extrapolation in dashboards

Distributed Tracing: End-to-End Visibility




```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```


Home › hrpoliciesAppInsight | Search
End-to-end transaction details
hrpoliciesAppInsight
451 search results
Agent conversation (Preview)
08:02:57 PM
hi, can you check the status of my order ord001? also, what's going on with ORD-002? i think i...
... Show more
You mentioned two order IDs: "ord001" and "ORD-002". I'll check the status for "ORD-001" and "ORD-002" since order IDs are case-sensitive and should be in the format ORD-001. - ORD-001 is Shipped - expected delivery in 2 days. - ORD-002 is Processing - not yet dispatched. If you need help with a different order ID or if you want me to check the status for "ord001" as entered, please let me know!
Show less
Search results
Learn more
Copy
Feedback
Enter simple view
End-to-end transaction
Operation ID: f6b9f7ed6d7d26ef0bb328f200c9b855
= Internal
= Dependency (outgoing)
= Traces & events occurrences
Search
EVENT
RES.
DURATION
invoke_agent
0
1.8 s
order-support-agent
invoke_agent
0
1.3 s
gpt-4.1-mini-2025-04-14
chat
0
1.2 s
Graph (Preview)
Highlight by: None
DEP
invoke_agent
1.8 s
AGENT
order-support-agent
1.3 s · 1.6k tokens
Traces & events 0 Traces 13 Events
View all
Create work item
order-support-agent
invoke_agent
Generative AI Properties
Show all
Event time
6/1/2026, 8:02:57.0049633 PM (Local time)
Duration
1.3 s
Messages
Copy
View full value
System
"""
Order-support agent demo with OpenTelemetry tracing into Application Insights.
Adapted to azure-ai-projects v2.x, which exposes agents through
User
hi, can you check the status of my order ord001? also, what's going on with ORD-002? i think i entered the order id wrong on one of them.
Generated output
You mentioned two order IDs: "ord001" and "ORD-002". I'll check the status for "ORD-001" and "ORD-002" since order IDs are case-sensitive and should be in the format ORD-001.
- ORD-001 is Shipped - expected delivery in 2 days.
Resource Properties
Show all
Compute platform
azure.aks

OpenTelemetry + Azure: The Modern Stack

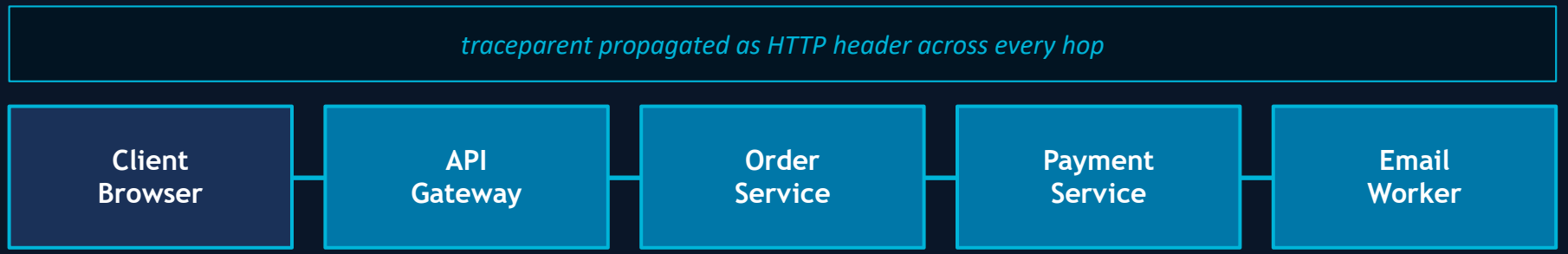
OTel SDK Instrumentation

1. Instrument	Auto-instrument ASP.NET Core, SQL, HTTP
2. Enrich	Add custom attributes: tenantId, region, version
3. Export	OTLP → Azure Monitor Exporter
4. Sample	Tail-based sampler on error/latency threshold

C# OTel Setup

```
builder.Services
  .AddOpenTelemetry()
  .WithTracing(b => b
    .AddAspNetCoreInstrumentation()
    .AddSqlClientInstrumentation()
    .AddHttpClientInstrumentation()
    .AddAzureMonitorTraceExporter(
      o => o.ConnectionString =
        config["AppInsights:CS"])
    .SetSampler(
      new TraceIdRatioBasedSampler(0.1)))
  .WithMetrics(b => b
    .AddAspNetCoreInstrumentation()
    .AddAzureMonitorMetricExporter());
```

Correlation ID Propagation Patterns



Critical Correlation Fields to Standardize

`operation_Id`	Root trace ID — identical across all telemetry for a single user request
`operation_ParentId`	Span's parent — enables true tree reconstruction
`customDimensions.correlationId`	Business key — order ID, tenant ID etc. for cross-log linking

KQL: Correlating Across Workspaces

1. Cross-workspace join

```
let infraLogs = workspace("infra-
ws")
  .Heartbeat | where TimeGenerated
> ago(1h);
let appLogs  = AppTraces
  | where operation_Id != "";
appLogs
| join kind=inner infraLogs
    on $left.operation_Id ==
$right.CorrelationId
```

2. Request → Dependency → Exception chain

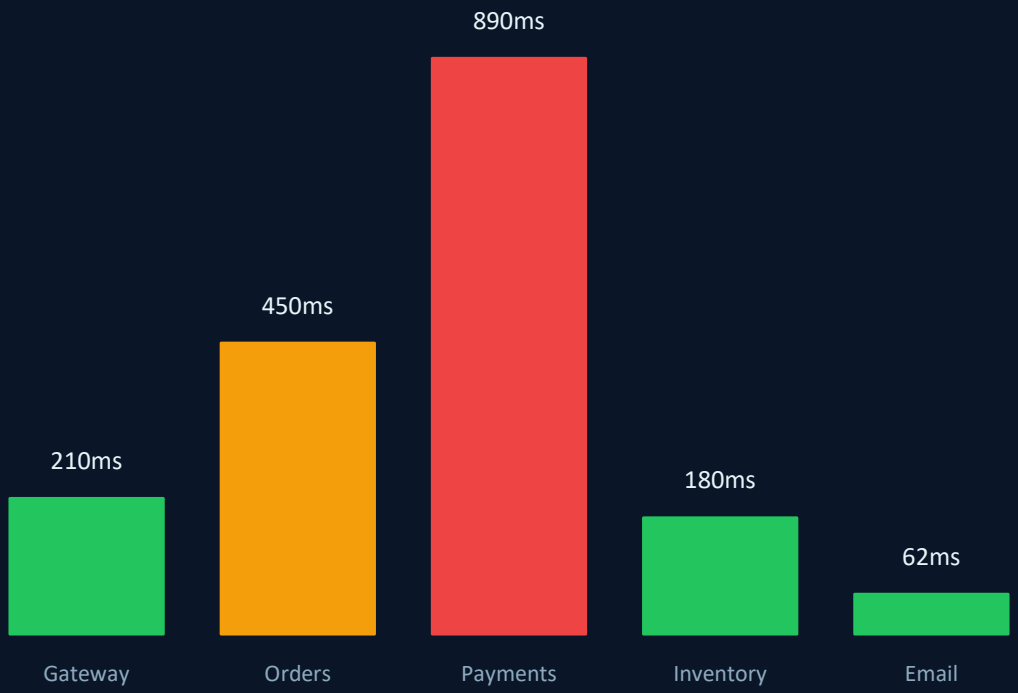
```
requests | where success == false
| extend tId = operation_Id
| join dependencies on $left.tId
    == $right.operation_Id
| join exceptions on $left.tId
    == $right.operation_Id
| project timestamp, name,
    type, outerMessage
```

3. P99 latency with trace context

```
requests
| where timestamp > ago(24h)
| summarize
    p99 = percentile(duration,
99),
    count = count()
    by bin(timestamp, 5m),
       cloud_RoleName
| order by p99 desc
```

Use workspace() + union to query multiple Log Analytics workspaces in a single KQL expression

Workbooks & Dashboards: Operationalizing Insights



Azure Workbook Capabilities

- ✓ Parametric time ranges & resource filters
- ✓ Conditional formatting on thresholds
- ✓ Drill-down from metric → trace → log
- ✓ Cross-resource query templates
- ✓ Exportable ARM templates for IaC
- ✓ Correlation heatmaps & dependency maps

Intelligent Alerting: Signal vs. Noise

P0 — Critical

Exception rate > 5% in 2 min rolling window
→ PagerDuty + War Room auto-create

P1 — High

P99 latency > 2× baseline for 5 min
→ Teams alert + auto-runbook triggered

P2 — Medium

Dependency failure rate > 1% over 15 min
→ Ticket created, on-call notified

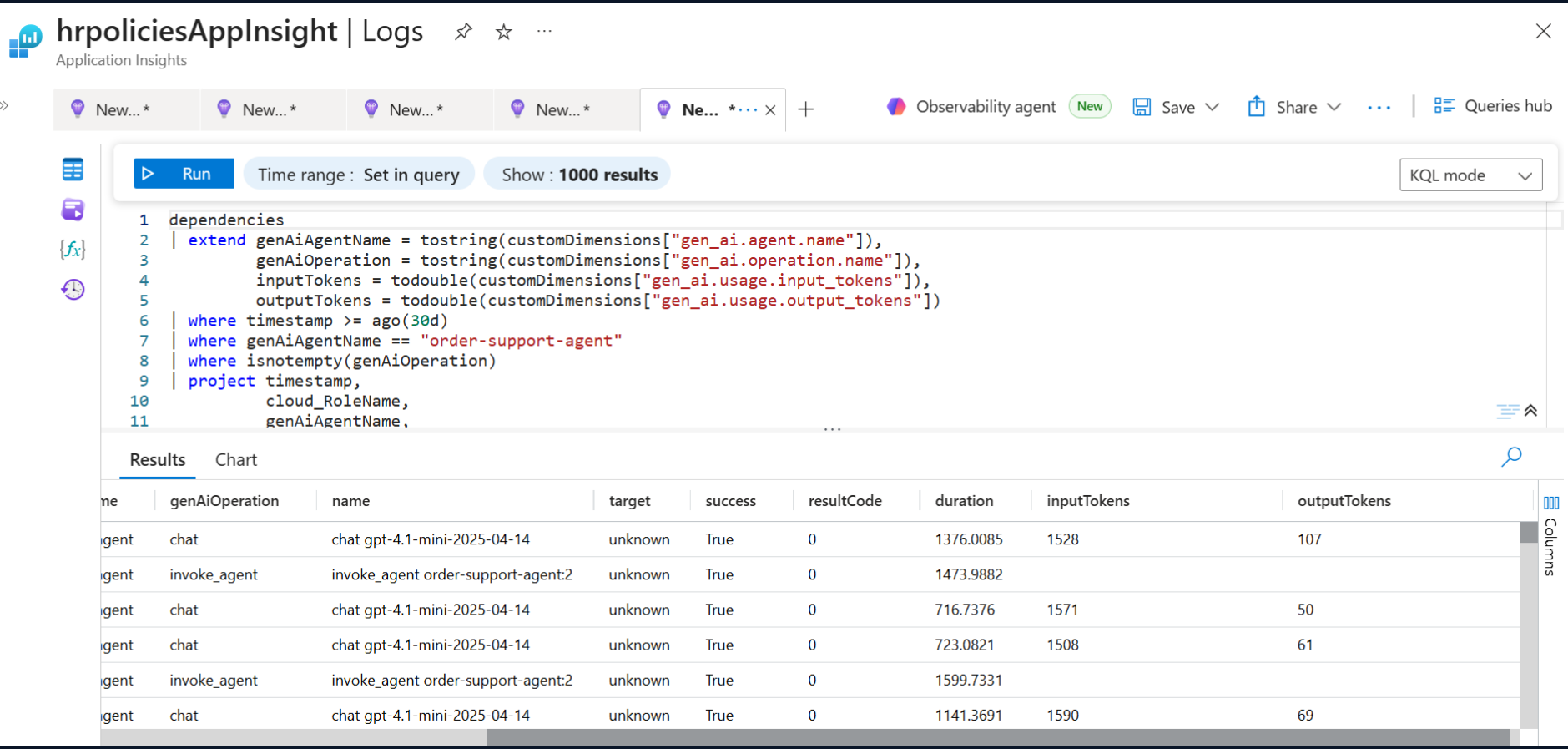
P3 — Low

Anomaly detection on business KPIs
→ Daily digest email summary

Best Practices: Dynamic baselines | Alert grouping to suppress flapping | Trace context in alert payload | Auto-silence during deployments

Production Patterns vs. Anti-Patterns

✓ DO	✗ DON'T
✓ Propagate W3C traceparent in all service calls	✗ Log unstructured text strings only
✓ Use structured logging (JSON) with operation_Id	✗ Use different correlation key names per team
✓ Tag every telemetry item with environment & version	✗ Leave default adaptive sampling at 5 req/s in prod
✓ Set Log Analytics data retention per table type	✗ Query across workspaces without resource tags
✓ Use private link for secure data ingestion	✗ Skip App Insights SDK for 'lightweight' services



me	genAiOperation	name	target	success	resultCode	duration	inputTokens	outputTokens
gent	chat	chat gpt-4.1-mini-2025-04-14	unknown	True	0	1376.0085	1528	107
gent	invoke_agent	invoke_agent order-support-agent:2	unknown	True	0	1473.9882		
gent	chat	chat gpt-4.1-mini-2025-04-14	unknown	True	0	716.7376	1571	50
gent	chat	chat gpt-4.1-mini-2025-04-14	unknown	True	0	723.0821	1508	61
gent	invoke_agent	invoke_agent order-support-agent:2	unknown	True	0	1599.7331		
gent	chat	chat gpt-4.1-mini-2025-04-14	unknown	True	0	1141.3691	1590	69

Observability Agent

Was this agent helpful? We'd love your feedback. Tell us what you think

Temporary chat

I also noticed you recently ran a KQL query. I can help interpret it, troubleshoot it, or refine it—would you like me to use it in this conversation?

Show me how many tokens were used by order-support-agent in last one month

Observability Agent

Reasoned for 0.38 minutes.

Summary

In the last 30 days, **`order-support-agent` used 138,986 tokens** total.

- **Input tokens:** 136,103
- **Output tokens:** 2,883
- **Dependency calls counted:** 87

This is based on dependency telemetry where the agent name and token usage fields were present.

Break down by dependency (30d) | Show daily token trend (30d) | Describe data sources and confidence

I want to...

AI-generated content may be incorrect

Key Takeaways

01 Azure Monitor + App Insights + Log Analytics form a unified observability platform — not three separate tools

02 operation_Id is your single source of truth — instrument it everywhere, propagate it always

03 OpenTelemetry is the future: vendor-neutral instrumentation exports natively to Azure Monitor

04 KQL cross-workspace joins unlock enterprise-scale log correlation across teams and services

05 Alerting on correlated signals (not individual metrics) dramatically reduces MTTR

Thank you!